

Tugas Kelompok – Berbasis Kasus 01

Robot Kurir Kampus: Rute GU (Gerbang Utama) → LK (Lab Komputasi)

Metode: UCS, IDS, GBFS, A* — Inteligensi Buatan 2026

Kelompok: Rx — Nama/NIM: [jisi nim1, nim2, nim3] — File ini draf otomatis dari output program **search_kampus.py**. Ganti nama file menjadi **TugasKelompok_Rx_nim1_nim2_nim3.pdf** sesuai kelompok.

1. Graf dan $h(n)$

Graf undirected 12 simpul, 19 edge (terverifikasi dari gambar hires). Tie-break alfabetis agar deterministik.

Simpul	Tetangga (bobot)
GU	PB(3), R(4)
PB	GU(3), GKU(4), K(6)
R	GU(4), PR(3), M(5)
GKU	PB(4), PR(2), K(3), IF(7)
K	PB(6), GKU(3), AS(5)
PR	R(3), GKU(2), A(6), IF(4)
A	PR(6), M(3), LK(5)
M	R(5), A(3), SC(6)
IF	GKU(7), PR(4), AS(4), LK(3)
AS	K(5), IF(4), SC(6)
SC	M(6), AS(6), LK(4)
LK	A(5), IF(3), SC(4)

Simpul	$h(n)$
GU	12
PB	10
R	9
GKU	7
PR	6
K	9
M	7
A	4
AS	5
IF	2
SC	3
LK	0

Heuristik diuji admissible dan konsisten ($h(n) \leq$ biaya aktual ke LK, dan $h(n) \leq c(n,m) + h(m)$ untuk semua edge). Contoh: $R(9) \leq 3 + 6 = 9$, $PR(6) \leq 4 + 2 = 6$, $GU(12) \leq 4 + 9 = 13$. Karena itu A* dengan h ini dijamin optimal pada graf ini.

2. Hasil luaran program (verbatim)

Perintah: **python search_kampus.py**. Di bawah ini salinan persis output yang dipakai sebagai bukti.

```
=====
KASUS 01: Robot kurir kampus GU -> LK
=====
```

```
[UCS] Urutan ekspansi (pop terkecil g):
GU -> PB -> R -> GKU -> PR -> K -> M -> IF -> A -> AS -> LK
g saat pop: {'GU': 0, 'PB': 3, 'R': 4, 'GKU': 7, 'PR': 7, 'K': 9, 'M': 9, 'IF': 11, 'A': 12, 'AS':
```

```

14, 'LK': 14}
Rute : GU -> R -> PR -> IF -> LK
Cost : 14 | node diekspansi: 11

[IDS] Iterative Deepening (DFS alfabetis, limit 0..n):
limit=0 (1 kunjungan): GU => belum ketemu
limit=1 (3 kunjungan): GU -> PB -> R => belum ketemu
limit=2 (7 kunjungan): GU -> PB -> GKU -> K -> R -> M -> PR => belum ketemu
limit=3 (17 kunjungan): GU -> PB -> GKU -> IF -> K -> PR -> K -> AS -> GKU -> R -> M -> A -> SC -
> PR -> A -> GKU -> IF => belum ketemu
limit=4 (6 kunjungan): GU -> PB -> GKU -> IF -> AS -> LK => KETEMU GU -> PB -> GKU -> IF -> LK
Rute : GU -> PB -> GKU -> IF -> LK
Cost (jumlah bobot) : 17 | depth solusi: 4 | total kunjungan kumulatif: 34

[GBFS] Urutan ekspansi (pop terkecil h):
GU -> R -> PR -> IF -> LK
Rute : GU -> R -> PR -> IF -> LK
Cost : 14 | node diekspansi: 5

[A*] Urutan ekspansi (pop terkecil f=g+h):
GU -> PB -> R -> PR -> IF -> GKU -> LK
detail pop (g,h,f): {'GU': (0,12,12), 'PB': (3,10,13), 'R': (4,9,13), 'PR': (7,6,13), 'IF': (11,2,
13), 'GKU': (7,7,14), 'LK': (14,0,14)}
Rute : GU -> R -> PR -> IF -> LK
Cost : 14 | node diekspansi: 7

--- RINGKASAN ---
UCS | ekspansi: 11 | rute: GU -> R -> PR -> IF -> LK | cost: 14
IDS | ekspansi: 34 (kumulatif, solusi depth 4) | rute: GU -> PB -> GKU -> IF -> LK | cost: 17
GBFS | ekspansi: 5 | rute: GU -> R -> PR -> IF -> LK | cost: 14
A* | ekspansi: 7 | rute: GU -> R -> PR -> IF -> LK | cost: 14

```

Verifikasi optimalitas (brute force semua simple path): jalur termurah adalah GU-R-PR-IF-LK = 4+3+4+3 = **14**. Runner-up GU-PB-GKU-PR-IF-LK = 3+4+2+4+3 = 16. Jadi cost 14 = optimal, cost 17 (IDS) = suboptimal.

3. Tabel perbandingan

Aspek	UCS	IDS	GBFS	A*
Prioritas	g(n)	depth (DFS)	h(n)	f=g+h
Ekspansi*	11	34 kum. (6 di limit 4)	5	7
Rute	GU-R-PR-IF-LK	GU-PB-GKU-IF-LK	GU-R-PR-IF-LK	GU-R-PR-IF-LK
Cost	14 optimal	17 suboptimal	14 (kebetulan optimal)	14 optimal
Edge	4	4	4	4

*Ekspansi = jumlah node di-pop. IDS menghitung ulang tiap limit (total 1+3+7+17+6=34). Rute IDS dangkal (depth 4) tapi mahal karena meminimalkan jumlah edge, bukan bobot.

4. Analisis perbandingan

Urutan ekspansi. UCS melebar berdasar biaya: GU(0), PB(3), R(4), GKU(7), PR(7), K(9), M(9), IF(11), A(12), AS(14), LK(14) — 11 node karena harus membuktikan tidak ada jalur <14. A* jauh lebih fokus: GU(12), PB(13), R(13), PR(13), IF(13), GKU(14), LK(14) — hanya 7 node, h memangkas cabang mahal (K, M, A, AS, SC tidak pernah di-pop). GBFS paling rakus: GU→R→PR→IF→LK (5 node), langsung mengikuti h menurun 12→9→6→2→0 tanpa menoleh ke cabang PB.

Rute dan cost. UCS dan A* menemukan optimal 14 (GU-R-PR-IF-LK). GBFS di kasus ini juga 14, tapi itu kebetulan karena h mengarah ke koridor optimal; secara teori GBFS tidak menjamin optimal (ia mengabaikan g). IDS menemukan GU-PB-GKU-IF-LK = 3+4+7+3 = 17 pada depth 4. Depth sama-sama 4 dengan solusi optimal, tetapi IDS berhenti pada solusi depth-4 pertama yang ditemui DFS alfabetis (cabang PB lebih dulu dari R), bukan yang termurah. Ini menjawab instruksi 'gunakan bobot untuk IDS': bobot tidak mengubah urutan DFS, hanya dipakai menghitung cost akhir.

Mengapa IDS boros? Setiap kenaikan limit mengulang seluruh pohon: limit 0,1,2,3 gagal (1+3+7+17 kunjungan) sebelum limit 4 berhasil. Total 34 kunjungan untuk graf 12 simpul. Kelebihannya hemat memori $O(d)$ dan tetap lengkap; kekurangannya waktu berlipat dan buta biaya. Cocok bila edge seragam, tidak cocok di sini karena bobot bervariasi 2–7.

Peran heuristik. Karena h admissible + konsisten (dicek per edge), A^* optimal sekaligus efisien (7 vs 11 ekspansi). GBFS (5 ekspansi) paling cepat di sini tetapi rapuh: jika h overestimate di satu simpul, ia bisa tersesat (misal bila $h(K)$ kecil, GBFS akan membelok ke K-AS-SC-LK yang mahal). UCS aman tanpa h tetapi membayar dengan ekspansi ekstra.

Kesimpulan praktis untuk robot kurir. Untuk peta kampus ini: pakai A^* (optimal + sedikit ekspansi). UCS sebagai baseline kebenaran. GBFS hanya bila butuh jawaban sangat cepat dan h terpercaya. IDS tidak disarankan untuk optimasi biaya; gunakan hanya bila memori sangat terbatas atau yang penting sedikit h transit (edge), bukan jarak meter.

5. Cara reproduce & lampiran kode

1) Simpan **search_kampus.py** dan PDF ini dalam satu folder. 2) Jalankan **python search_kampus.py** — output harus sama persis dengan Bab 2. 3) Tie-break alfabetis, DLS memakai path-checking (hindari cycle satu jalur), UCS/GBFS/ A^* memakai graph-search. 4) Ganti placeholder Rx/NIM di cover, lalu export/rename final menjadi **TugasKelompok Rx_nim1_nim2_nim3.pdf**. Kode lengkap ada di file pendamping; ringkasan struktur: dict GRAPH + H , fungsi `ucs()`, `dls()/ids()`, `gbfs()`, `astar()`, `path_cost()`.

Draf ini belum final: tambahkan nama anggota, NIM, kelas, screenshot terminal, dan gambar graf kampus sebelum dikumpulkan.